Doc. no.:	P0608R3
Date:	2018-10-03
Audience:	LWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

A sane variant converting constructor

Changes since R1

- Discuss whether to add special cases for unambiguous conversions.
- Take the user-defined types that trigger standard conversions into account.
- Remove an unhelpful example.

Problems to solve

1. `variant` constructs entirely unintended alternatives.

```
variant<string, bool> x = "abc";             // holds bool
```

The above holds `string` with the proposed fix.

2. `variant` prefers constructions with information losses.

```
variant<char, optional<char16_t>> x = u'\u2043';  // holds char = 'C'
double d = 3.14;
variant<int, reference_wrapper<double>> y = d;    // holds int = 3
```

The above preserves the input value in `optional<char16_t>` and `reference_wrapper<double>`, respectively, with the proposed fix.

3. `variant` performs unstable constructions.

```
using T = variant<float, int>;
T v;
v = 0;    // switches to int
```

- When `T` is upgraded to `variant<float, long>`,

```
using T = variant<float, long>;
T v;
v = 0;    // error
```

- When `T` is upgraded to `variant<float, big_int<256>>`,

```
using T = variant<float, big_int<256>>;
T v;
v = 0;    // holds 0.f
```

In both cases, the proposed fix consistently constructs with the second alternative.

As shown, the problems equally apply to the converting constructor and the converting assignment operator.

See also LEWG 227 (https://issues.isocpp.org/show_bug.cgi?id=227).

Proposed resolution

This paper proposes to constrain the `variant` converting constructor and the converting assignment operator to prevent narrowing conversions and conversions to `bool`. This section explains what exactly this change brings.

Lemma 1. Let X be a sum type $T + U$. Let $\mathrm{A} = \{T_1, T_2, T_3, \dots\}$be a set of types that are convertible to T, and $\mathrm{B} = \{U_1, U_2, U_3, \dots\}$be a set of types that are convertible to U. Let Y be a set of types that are convertible to X. $\mathrm{Y} = \mathrm{A} \ominus \mathrm{B}$(symmetric difference) rather than $\mathrm{A} \cup \mathrm{B}$, because each $T_i = U_j$ causes an ambiguity.

Theorem 1. If A and B are extended to include a type $\tau \in \mathrm{A} \cap \mathrm{B}$, Y is shrunk.

In short, constraining `variant<` $Ts...$ `>` (with $\overline{Ts} > 1$) converting constructor may enable more types to be convertible to a variant.

Definition 1. For type T that is convertible to T', let P be a set of all the possible values for T, and Q be a set of all the possible values for T'. If $\mathrm{P} \subseteq \mathrm{Q}$, T' is denoted as T^+. Otherwise, $\mathrm{P} \nsubseteq \mathrm{Q}$ and T' is denoted as T^-. The conversion from T to T^- (denoted as $T \rightharpoonup T^-$) is a *potentially unrepresentable* conversion.

In this paper, narrowing conversions (considering only the types) and boolean conversions assemble the *potentially unrepresentable* conversions in C++.

Lemma 2. *Potentially unrepresentable* conversions in C++ have Conversion rank.

The proof is left as an exercise for the reader.

Let `X` be `variant<` $Ts...$ `>`, `r` be a value of T.

When $\overline{Ts} = 1$, without loss of generality, `X` is `variant<` T^- `>`. Effects of the proposed resolution can be summarized as follows:

`X v = r;`	before	after
`variant<float> v = 0;`	holds `.0f`	ill-formed
`variant<float> v = INT_MAX;`	holds `INT_MAX + 1`	ill-formed

However, `variant<V>` is such a rare variant, as you can hardly say that $V + \bot$ is a sum type.

When $\overline{Ts} = 2$, `X` is `variant<` T^-, U `>`.

- If T is not convertible to U, the effects are the same as the case $\overline{Ts} = 1$.
- If $T \rightharpoonup U$ is an Exact Match or a Promotion, $T \rightharpoonup T^-$ has a worse conversion sequence, U is constructed in `X v = r` under the existing rule. The proposed resolution constructs U as well because $T \rightharpoonup T^-$ is not viable.
- If T is potentially unrepresentable converted to U, according to Lemma 2, both $T \rightharpoonup U$ and $T \rightharpoonup T^-$ have Conversion rank, so `X v = r` is ill-formed due to ambiguity under the existing rule. Meanwhile, it's also ill-formed given the proposed resolution, because both conversions are potentially unrepresentable.
- If $T \rightharpoonup U$ has Conversion rank and is not potentially unrepresentable, `X v = r` is still ill-formed under the existing rule. However, U is constructed given the proposed resolution.
- If $T \rightharpoonup U$ is a user-defined conversion, `X v = r` constructs T^- under the existing rule. With the proposed resolution, it constructs U instead.

The effects are summarized in the order of the bullets:

`X v = r;`	before	after
`variant<float, vector<int>> v = 0;`	holds `float`	ill-formed
`variant<float, int> v = 'a';`	holds `int('a')`	holds `int('a')`
`variant<float, char> v = 0;`	ill-formed	ill-formed
`variant<float, long> v = 0;`	ill-formed	holds `long`
`variant<float, big_int<256>> v = 0;`	holds `float`	holds `big_int`

When $\overline{Ts} > 2$, let $Ts_1 = T^-$, S be an overload set $\{f(\tau) \mid \tau \in Ts\}$ $S' = S - \{f(T^-)\}$.

- If $S(r)$ resolves to $f(T^-)$, this is equivalent to the case $\overline{Ts} = 1$; or
- if $S'(r)$ resolves to $f(U)$ where $T \rightharpoonup U$ has Conversion rank, `X v = r` is ill-formed without the proposed resolution; or
- if $S'(r)$ resolves to $f(U)$ where $T \rightharpoonup U$ is a user-defined conversion, $S' - \{f(U)\}$ has no viable function against r, and the situation is equivalent to the last bullet in the case $\overline{Ts} = 2$.
- Otherwise, $S'(r)$ resolves to $f(U)$ where $T \rightharpoonup U$ is an Exact Match or a Promotion, `X v = r` constructs U with or without the proposed resolution.

The effects are summarized in the order of the bullets:

`X v = r;`	before	after
`variant<float, big_int<256>, big_int<128>> v = 0;`	holds `float`	ill-formed
`variant<float, long, double> v = 0;`	ill-formed	holds `long`
`variant<float, vector<int>, big_int<256>> v = 0;`	holds `float`	holds `big_int`
`variant<float, int, big_int<256>> v = 'a';`	holds `int`	holds `int`

Theorem 2. For `variant<` $Ts...$`> v = r` , where `r` is a value of R, when there exists one and only one $\tau \in Ts$ rendering R to be potentially unrepresentable converted to τ, the proposed resolution may cause breaking changes

- by preventing the initialization if it used to construct τ, or
- by preferring a different and user-defined conversion $R \rightharpoonup U$ when there is exactly one $U \in Ts - \{\tau\}$ that is convertible from R.

The first case is easy to fix while the second gives the desired outcome for this paper. Both behaviors to be changed are bugs, not features, as shown in Section 1.

However, the definition of *potentially unrepresentable* conversions here makes it unable to be fully detected in the library, because while we understand type conversions as mappings, C++ type conversions happen in conversion sequences. A boolean conversion in a standard conversion sequence that follows a user-defined conversion sequence cannot be detected. In short, we cannot distinguish

```
struct Bad { operator char const*(); };
```

from

```
struct LessBad { operator bool(); };
```

The proposed resolution considers both to be bad. Throughout the library, only two types are affected: `true_type` and `false_type` . The most commonly seen cases, types that are contextually converted to `bool` , are not affected by this compromise, because `std::variant` is

not contextual. The compromise also does not break our proof, because user-defined conversions' rank is lower than that of narrowing conversions.

Alternative designs

The author came up with and experimented a few other designs, here we list two basic ideas.

1. Use the alternatives' order information in determining which one to construct. The idea defeats the purpose of the converting constructor because if the construction is sensitive to the order of the alternatives declared in the `variant` template argument list, `in_place_index` would be a better choice. The converting constructor and assignment operator assume unordered alternatives.
2. Distinguish implicit and explicit conversions. First, the idea doesn't work well with the converting assignment operator; applying the implicit policy seems to be the only choice to maintain a consistent behavior, but this may be overkill. Second, it is counterintuitive to have an explicit constructor accepting fewer types comparing to an implicit one because of Theorem 1.

An additional suggestion is to allow narrowing but unambiguous conversions. In the following example, if `double_double` is a user-defined type that does not convert from an integer type,

```
variant<char, double_double> v = 42;
```

, it might be not so problematic to construct the `char` alternative. However, special casing this makes the construction less stable. If `double_double` is upgraded to support converting from `int`, the code silently constructs the `double_double` alternative.

Wording

This wording is relative to N4762.

Modify 19.7.3.1 [variant.ctor]/12 as indicated:

```
template<class T> constexpr variant(T&& t) noexcept(see below );
```

> Let T_j be a type that is determined as follows: build an imaginary function *FUN*`(`T_i`)` for each alternative type T_i for which `T`$_i$ `x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` and, if T_i is *cv* `bool`, `remove_cvref_t<T>` is `bool`. The overload *FUN*`(`T_i`)` selected by overload resolution for the expression *FUN*`(std::forward<T>(t))` defines the alternative T_j which is the type of the contained value after construction.

> […]

[Drafting note: The above uses `T`$_i$ `x[] = {std::forward<T>(t)};` rather than `T`$_i$ `{std::forward<T>(t)}` to test whether the conversion sequence from `T` to `T`$_i$ contains a narrowing conversion because whether `T`$_i$ `{std::forward<T>(t)}` is well-formed may subject to whether `T`$_i$ has an `initializer_list` constructor or whether `T`$_i$ is an aggregate. Here is an example: https://godbolt.org/z/Ck5w-L (https://godbolt.org/z/Ck5w-L) *–end note]*

Modify 19.7.3.3 [variant.assign]/10 as indicated:

```
template<class T> variant& operator=(T&& t) noexcept(see below );
```

Let T_j be a type that is determined as follows: build an imaginary function *FUN*`(`T_i`)` for each alternative type T_i for which `T`$_i$` x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x` and, if `T`$_i$ is *cv* `bool`, `remove_cvref_t<T>` is `bool`. The overload *FUN*`(`T_i`)` selected by overload resolution for the expression *FUN*`(std::forward<T>(t))` defines the alternative T_j which is the type of the contained value after assignment.

[…]